

<https://github.com/MrRogueKnight/Dev3Pack-Lisk-Solidity-Bootcamp-2025/tree/rogue/homework/Week%20Lecture%20-%20Solidity%20Fundamentals>

Homework Completed

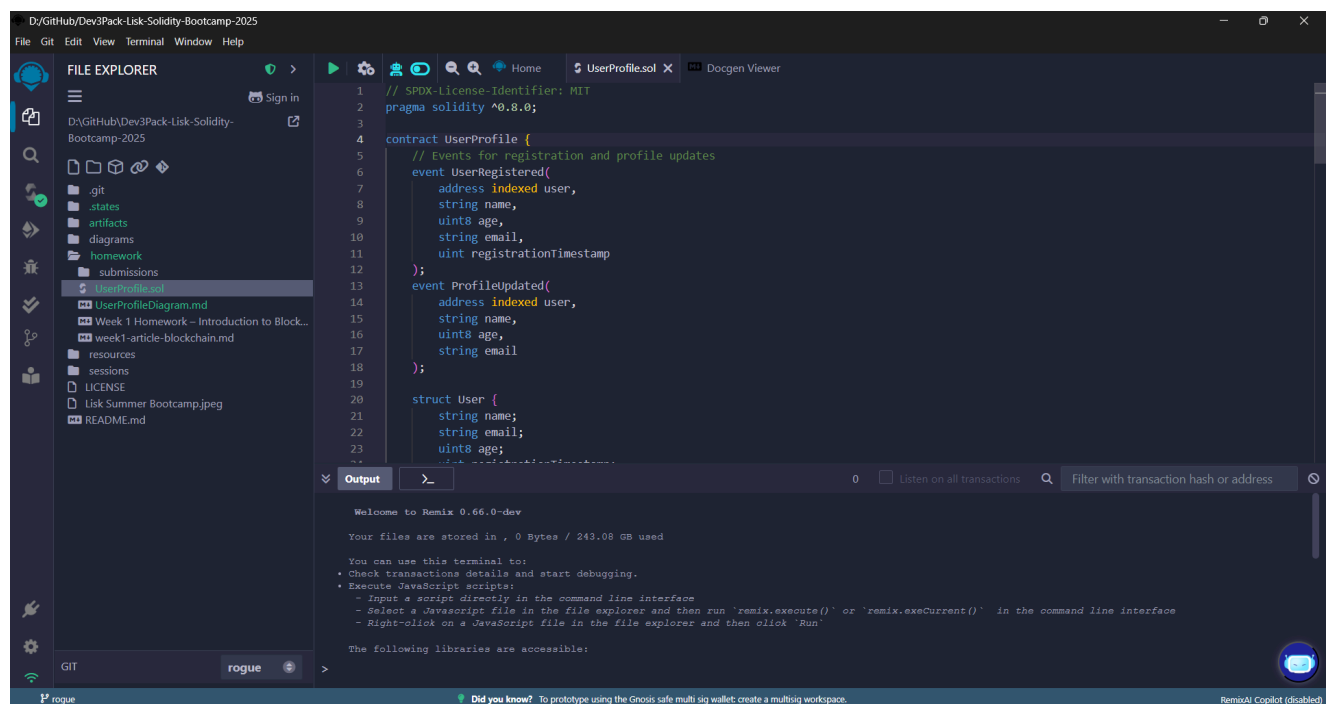
I have completed the assignment for **Week 1 Lecture 2 – Solidity Fundamentals**.

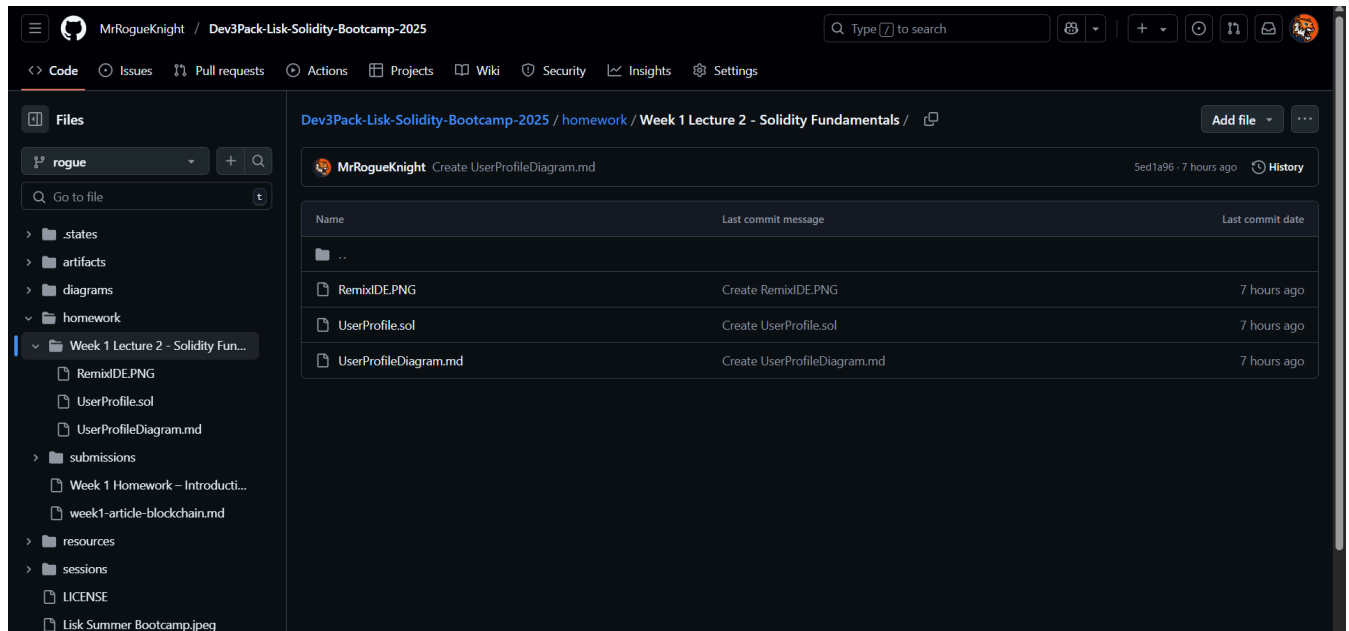
The smart contract includes all required functions: `register()`, `updateProfile()`, and `getProfile()`.

It uses a `User` struct, validates single registration, tracks timestamps, and includes proper state storage.

 **GitHub Repository:**

<https://github.com/MrRogueKnight/Dev3Pack-Lisk-Solidity-Bootcamp-2025/tree/rogue/homework/Week%20Lecture%20-%20Solidity%20Fundamentals>





<https://github.com/MrRogueKnight/Dev3Pack-Lisk-Solidity-Bootcamp-2025/tree/rogue/homework/Week%201%20Lecture%202%20-%20Solidity%20Fundamentals>

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract UserProfile {
    // Events for registration and profile updates
    event UserRegistered(
        address indexed user,
        string name,
        uint8 age,
        string email,
        uint registrationTimestamp
    );
    event ProfileUpdated(
        address indexed user,
        string name,
        uint8 age,
        string email
    );

    struct User {
        string name;
```

```

    string email;
    uint8 age;
    uint registrationTimestamp;
}

mapping(address => User) private users;
mapping(address => bool) private registered;

function register(
    string calldata name,
    uint8 age,
    string calldata email
) external {
    require(!registered[msg.sender], "User already registered");
    require(bytes(name).length > 0, "Name cannot be empty");
    require(bytes(email).length > 0, "Email cannot be empty");
    require(age <= 150, "Unrealistic age");
    users[msg.sender] = User({
        name: name,
        email: email,
        age: age,
        registrationTimestamp: block.timestamp
    });
    registered[msg.sender] = true;
    emit UserRegistered(msg.sender, name, age, email, block.timestamp);
}

function updateProfile(
    string calldata name,
    uint8 age,
    string calldata email
) external {
    require(registered[msg.sender], "User not registered");
    require(bytes(name).length > 0, "Name cannot be empty");
    require(bytes(email).length > 0, "Email cannot be empty");
    require(age <= 150, "Unrealistic age");
    users[msg.sender].name = name;
    users[msg.sender].email = email;
    users[msg.sender].age = age;
    emit ProfileUpdated(msg.sender, name, age, email);
}

function getProfile()
    external

```

```

view
returns (
    string memory name,
    uint8 age,
    string memory email,
    uint registrationTimestamp
)
{
    require(registered[msg.sender], "User not registered");
    User memory user = users[msg.sender];
    return (user.name, user.age, user.email, user.registrationTimestamp);
}
}

```

/*

Explanations:

- Added input validation for name, email, and age fields in register and updateProfile.
- Added events: UserRegistered and ProfileUpdated, emitted on registration and profile update.
- Used external visibility for functions that are not called internally.
- Reordered struct fields for better storage packing (string, string, uint8, uint).

*/